Lab 04: Kullanıcı Girdileri ve Hareket

PyGame ile Oyun Geliştirme – Hafta 4

Ad Soyad: Mehmet Mert Kurt	Numara: 439241	Tarih:

1 Çalışma 1: Event Gözlemcisi

Örnek Kod

Aşağıdaki programı `olay_gozlemci.py` dosyasına yazın ve çalıştırın:

```python
import pygame
import sys

pygame.init()

GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Olay Gozlemci")
saat = pygame.time.Clock()

KOYU_GRI = (40, 40, 40)

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        else:
            # Tum olaylari konsola yazdir
            print(f"Olay: {olay}")

    ekran.fill(KOYU_GRI)
    pygame.display.flip()
    saat.tick(60)

pygame.quit()
sys.exit()
```

`event.get()` Her Turda Çağrılmalı

`pygame.event.get()` fonksiyonu olay kuyruğunu boşaltır ve bekleyen tüm olayları bir liste olarak döndürür. Her oyun döngüsünde mutlaka çağrılmalıdır. Çağrılmadığı takdirde kuyruk dolar ve pencere yanıt vermez hale gelir.

Talep 1: Olayları Gözlemle

Kodu çalıştırın, farklı tuşlara basın ve fare ile tıklayın. Terminalde gözlemlenen olay türlerini aşağıya yazın:

Klavyeye basınca	Olay:<Event(768-KeyDown {'unicode': 'a', 'key': 97, 'mod': 4096, 'scancode': 4, 'window': None})> Olay:<Event(771-TextInput {'text': 'a', 'window': None})>
Klavyeyi bırakınca	Olay:<Event(769-KeyUp {'unicode': 'a', 'key': 97, 'mod': 4096, 'scancode': 4, 'window': None})>
Fare tıklayınca	Olay:<Event(1025-MouseButtonDown {'pos': (770, 579), 'button': 1, 'touch': False, 'window': None})> Olay:<Event(1026-MouseButtonUp {'pos': (770, 579), 'button': 1, 'touch': False, 'window': None})>
Fare hareket edince	Olay:<Event(1024-MouseMotion {'pos': (665, 599), 'rel': (4, 6), 'buttons': (0, 0, 0), 'touch': False, 'window': None})>
Pencere kapatılınca	Olay:<Event(32787-WindowClose {'window': None})>

Talep 2: Olay Filtresi

Olay işleme bloğunu düzenleyerek yalnızca `KEYDOWN` ve `MOUSEBUTTONDOWN` olaylarını yazdırın, diğerlerini göz ardı edin.
İpucu: `if` ifadesiyle olay türünü kontrol edin:

```
if olay.type == pygame.KEYDOWN:
    print(f"Basildi: {pygame.key.name(olay.key)}")
elif olay.type == pygame.MOUSEBUTTONDOWN:
    print(f"Fare: dugme={olay.button}, konum={olay.pos}")
```

2 Çalışma 2: Klavye ile Hareket

Örnek Kod

Aşağıdaki programı `klavye_hareket.py` dosyasına yazın ve çalıştırın:

```
import pygame
import sys

pygame.init()

GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Klavye ile Hareket")
saat = pygame.time.Clock()
FPS = 60

LACIVERT = (20, 30, 70)
SARI     = (255, 220, 50)

# Kare özellikleri
kare_x = GENISLIK // 2
kare_y = YUKSEKLIK // 2
kare_bov = 40
HIZ = 200  # piksel/saniye

çalıştır = True
while çalıştır:
    dt = saat.tick(FPS) / 1000.0  # saniye cinsinden delta time

```

```python
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        elif olay.type == pygame.KEYDOWN:
            if olay.key == pygame.K_ESCAPE:
                çalıştır = False

    # Surekli girdi kontrolu (get_pressed)
    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_UP]:
        kare_y -= HIZ * dt
    if tuslar[pygame.K_DOWN]:
        kare_y += HIZ * dt
    if tuslar[pygame.K_LEFT]:
        kare_x -= HIZ * dt
    if tuslar[pygame.K_RIGHT]:
        kare_x += HIZ * dt

    ekran.fill(LACIVERT)
    pygame.draw.rect(ekran, SARI,
        (int(kare_x), int(kare_y), kare_bov, kare_bov))
    pygame.display.flip()

pygame.quit()
sys.exit()
```

Sürekli Girdi vs. Ayrık Girdi Farkı

İki farklı girdi okuma yöntemi vardır:

- `pygame.key.get_pressed()` → **Sürekli girdi**: Tuş basılı tutulduğu sürece her karede `True` döner. Hareket için idealdir.
- `KEYDOWN` olayı → **Ayrık girdi**: Tuşa yalnızca bir kez basıldığında tetiklenir. Zıplama, ateş etme gibi tek seferlik eylemler için kullanın.

Talep 1: Hız Deneyi

`HIZ` sabitini sırasıyla `100`, `300` ve `500` olarak değiştirip programı çalıştırın. Her hız değerindeki davranış farkını gözlemleyin ve aşağıya not edin:

HIZ = 100	Sar kare daha yava hareket ediyor.
HIZ = 300	Sar kare daha hzl hareket ediyor.
HIZ = 500	Sar kare çok daha hzl hareket ediyor.

Talep 2: WASD Desteği

Ok tuşlarının yanı sıra WASD klavye düzenini de destekleyin:

- `K_w` → Yukarı
- `K_a` → Sol
- `K_s` → Aşağı
- `K_d` → Sağ

Talep 3: Renk Değişimi

Space tuşuna basıldığında karenin rengi değişsin. `KEYDOWN` olayını kullanarak her basışta farklı bir renk seçin:

```
renkler = [(255,220,50), (50,200,100), (50,150,255), (255,100,100)]
renk_indis = 0

# KEYDOWN olayinda:
if olay.key == pygame.K_SPACE:
    renk_indis = (renk_indis + 1) % len(renkler)
```

3 Çalışma 3: Fare ile Etkileşim

Örnek Kod

Aşağıdaki programı `fare_etkilesim.py` dosyasına yazın ve çalıştırın:

```
import pygame
import sys

pygame.init()

GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Fare ile Etkilesim")
saat = pygame.time.Clock()

KOYU_MAVI = (15, 25, 60)
MAVI      = (50, 120, 255)

daireler = []  # (x, y) pozisyonlarini tut

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False
        elif olay.type == pygame.MOUSEBUTTONDOWN:
            if olay.button == 1:  # Sol tikla
                daireler.append(olay.pos)
```

```

    ekran.fill(KOYU_MAVI)

    for konum in daireler:
        pygame.draw.circle(ekran, MAVI, konum, 20)

    pygame.display.flip()
    saat.tick(60)

pygame.quit()
sys.exit()
```

Talep 1: Tıklama Rengi

Sol tıklama mavi (`(50, 120, 255)`), sağ tıklama kırmızı (`(220, 60, 60)`) daire oluştursun. Listeye renk bilgisini de ekleyin:

```
# (x, y, renk) uclusu olarak sakla
daireler.append((olay.pos[0], olay.pos[1], MAVI))
```

Sağ tıklama için `olay.button == 3` koşulunu kullanın.

Talep 2: Fareyi Takip Eden Nesne

`pygame.mouse.get_pos()` ile fare konumunu her karede okuyun ve fareyi takip eden küçük bir daire çizin:

```
# Güncelleme ve cizim bolumunde:
fare_x, fare_y = pygame.mouse.get_pos()
pygame.draw.circle(ekran, (255, 255, 100),
    (fare_x, fare_y), 12)
```

Talep 3: Daireleri Temizle

`K_c` tuşuna basıldığında tüm daireler silinsin. `KEYDOWN` olayında listeyi boşaltın:

```
if olay.type == pygame.KEYDOWN:
    if olay.key == pygame.K_c:
        daireler.clear()
```

4 Çalışma 4: Sınır Kontrolü

Örnek Kod

Aşağıdaki programı `sinir_kontrol.py` dosyasına yazın ve çalıştırın:

```
import pygame
import sys

pygame.init()

GENISLIK = 800
```

```python
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Sinir Kontrolu - Clamp")
saat = pygame.time.Clock()
FPS = 60

LACIVERT  = (15, 25, 60)
TURUNCU   = (255, 160, 20)

# Top özellikleri
top_x  = 100.0
top_y  = 100.0
top_r  = 25        # yaricap
hiz_x  = 250.0     # piksel/saniye
hiz_y  = 180.0

çalıştır = True
while çalıştır:
    dt = saat.tick(FPS) / 1000.0

    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    # Hareketi güncelle
    top_x += hiz_x * dt
    top_y += hiz_y * dt

    # Clamp: sinir icerisinde tut
    top_x = max(top_r, min(top_x, GENISLIK - top_r))
    top_y = max(top_r, min(top_y, YUKSEKLIK - top_r))

    ekran.fill(LACIVERT)
    pygame.draw.circle(ekran, TURUNCU,
        (int(top_x), int(top_y)), top_r)
    pygame.display.flip()

pygame.quit()
sys.exit()
```

Üç Sınır Stratejisi

Bir nesne ekranın kenarına ulaştığında üç farklı strateji uygulanabilir:

- **Clamp (Kilitleme):** Nesneyi kenarda durdur. `x = max(r, min(x, GEN - r))`
- **Bounce (Sekme):** Kenara çarpınca hız tersine döner. `if x > GEN - r: hiz_x = -hiz_x`
- **Wrap (Sarmalama):** Bir kenardan çıkan nesne karşı kenardan geri girer. `if x > GEN: x = -r`

Talep 1: Bounce Stratejisi

Clamp yerine Bounce (sekme) stratejisini uygulayın. Top duvara çarptığında ilgili hız bileşenini tersine çevirin:

```
if top_x - top_r < 0 or top_x + top_r > GENISLIK:
    hiz_x = -hiz_x
if top_y - top_r < 0 or top_y + top_r > YUKSEKLIK:
    hiz_y = -hiz_y
```

Talep 2: Wrap Stratejisi

Wrap (sarmalama) stratejisini uygulayın. Top sağ kenardan çıkınca sol kenardan, alt kenardan çıkınca üst kenardan tekrar girsin:

```
if top_x - top_r > GENISLIK:
    top_x = -top_r
elif top_x + top_r < 0:
    top_x = GENISLIK + top_r
if top_y - top_r > YUKSEKLIK:
    top_y = -top_r
elif top_y + top_r < 0:
    top_y = YUKSEKLIK + top_r
```

Talep 3: Üç Top

Üç topu aynı anda farklı stratejilerle hareket ettirin:

- **Turuncu top:** Clamp – kenarda durur
- **Mavi top:** Bounce – sekme yapar
- **Yeşil top:** Wrap – karşı kenardan girer

Her top için ayrı `x`, `y`, `hiz_x`, `hiz_y` değişkenleri tanımlayın.

Bonus: Mini Yılan Prototipi

Öğrendiklerinizi birleştirerek yılan oyununun basit bir prototipini yazın:
Kurallar:

- Izgara tabanlı hareket – yılan her adımda bir hücre ilerler
- Ok tuşlarıyla yön değiştirme (`KEYDOWN` ile)
- Kırmızı bir yem rastgele konumda belirir
- Yem yenince yılan bir hücre büyür (kuyruğa yeni segment eklenir)

Temel Yapı:

```
HUCRE = 20            # piksel cinsinden hücre boyutu
COLS  = 800 // HUCRE
ROWS  = 600 // HUCRE

# Yilani liste olarak tut: [(sutun, satir), ...]
yilan = [(10, 10), (9, 10), (8, 10)]
yon   = (1, 0)        # (dx, dy): sag

# Her hareket adiminda:
yeni_bas = (yilan[0][0] + yon[0],
            yilan[0][1] + yon[1])
yilan.insert(0, yeni_bas)  # Basa yeni konum ekle
# Yem yenmediyse kuyrugu kaldir:
yilan.pop()
```

Adimlar:

1. Izgara arka planini ciz (isteğe bagli)
2. Yilani yeşil kareler olarak ciz
3. Yemi kirmizi kare olarak ciz
4. Basa girdi isleme ekle (K_UP, K_DOWN, K_LEFT, K_RIGHT)
5. Yeni bas yeme esitse: `yilan.pop()` cagirma ve yeni yem oluştur
6. Basi duvara veya kuyruga carpiyorsa oyunu sifirla

Ipucu: Yilan hizini kontrol etmek için her N karede bir adim atmak uzere bir sayac kullanin:

```
ADIM_KAREDE = 10    # kac karede bir adim atilacagi
sayac = 0

# Dongu icinde:
sayac += 1
if sayac >= ADIM_KAREDE:
    sayac = 0
    # Yilani hareket ettir
```

Başarılar!